

P.PORTO

Angular – Autenticação e Bibliotecas

Programação em Ambiente Web

Índice

Autenticação & Autorização

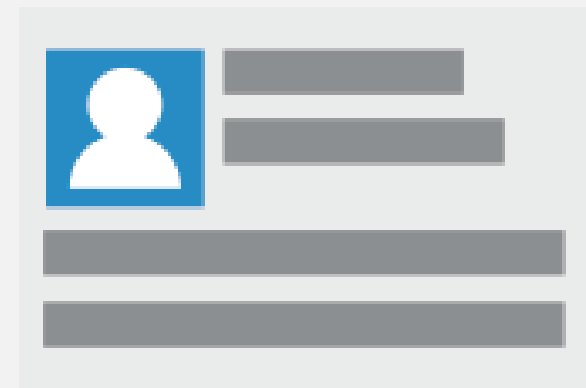
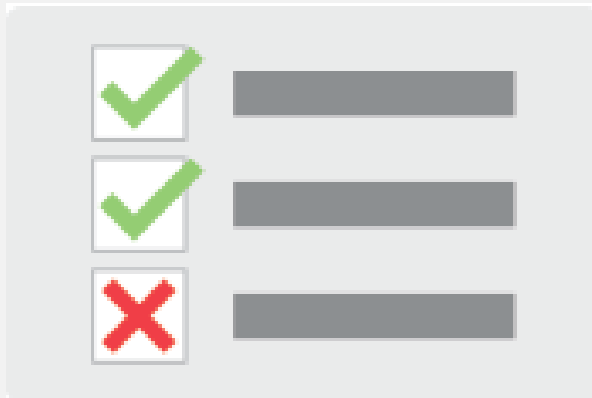
NodeJS e ExpressJS - Autenticação & Autorização

Angular - Autenticação & Autorização

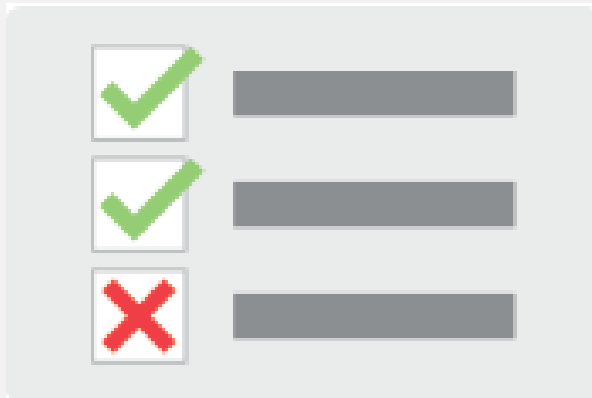
Angular Material

Angular Components

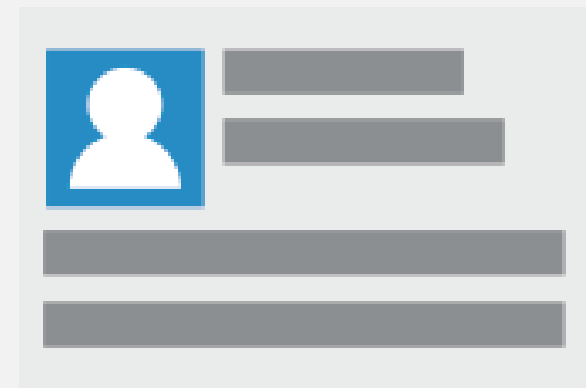
Autenticação & Autorização



Autenticação & Autorização



Autorização



Autenticação

Autenticação & Autorização

Autenticação e autorização são dois conceitos diferentes em aplicações web

Autenticação diz respeito a identificação do utilizador, deve assegurar-nos que o utilizador é quem diz ser

Autorização diz respeito às permissões de um utilizador na aplicação (ex: utilizadores admin e guest)

Autenticação & Autorização

Podemos na nossa aplicação em ExpressJS definir uma autenticação básica usando middleware:

- Enviar para o servidor informação de login em http headers
- Verificar no servidor as informações nos headers e fornecer acesso aos recursos
- Dar a resposta ao cliente com os recursos autorizados

O transporte da informação http em ambientes de produção deve ser feito através do protocolo HTTPS

Autenticação & Autorização

Do lado do cliente devemos enviar a informação de login e esperar do servidor a confirmação de acesso aos recursos

Devemos perceber que apesar de estarmos autenticados podemos não ter acesso aos recursos e por isso devemos adoptar mecanismos de captura de erros

NodeJS e ExpressJS - Autenticação & Autorização

Partindo do ponto de partida de uma REST API vamos demonstrar o uso de autenticação e autorização através de tokens JWT

Instalar o módulo jsonwebtoken:

- `npm install --save jsonwebtoken`

NodeJS e ExpressJS - Autenticação & Autorização

Definir um segredo para autenticação via jwt:

- Ficheiro authconfig.js

```
module.exports = {  
  'secret': 'supersecret'  
};
```

NodeJS e ExpressJS - Autenticação & Autorização

Realizar a autenticação de um utilizador guardado na base de dados

Modelo de um utilizador na base de dados

```
var mongoose = require('mongoose');
var UserSchema = new mongoose.Schema({
  name: String,
  email: String,
  password: String
});
mongoose.model('User', UserSchema);

module.exports = mongoose.model('User');
```

NodeJS e ExpressJS - Autenticação & Autorização

Controlador de
usado para gerir a
autenticação e
autorização

```
var User = require('../models/user');

var jwt = require('jsonwebtoken'); // used to create, sign, and verify tokens
var bcrypt = require('bcryptjs');
var config = require('../authconfig'); // get config file

//var User = require('./User');
var authController = {};

authController.login = function(req, res) {
  User.findOne({ email: req.body.email }, function (err, user) {
    if (err) return res.status(500).send('Error on the server.');
```

if (!user) return res.status(404).send('No user found.');

// check if the password is valid

var passwordIsValid = bcrypt.compareSync(req.body.password, user.password);

if (!passwordIsValid) return res.status(401).send({ auth: false, token: null });

// if user is found and password is valid

// create a token

var token = jwt.sign({ id: user._id }, config.secret, {
 expiresIn: 86400 // expires in 24 hours
});

// return the information including token as JSON

res.status(200).send({ auth: true, token: token });

});

}

authController.logout = function(req, res) {

res.status(200).send({ auth: false, token: null });

}

NodeJS e ExpressJS - Autenticação & Autorização

Controlador de
usado para gerir a
autenticação e
autorização

(continuação..)

```
authController.register = function(req, res) {  
  
  var hashedPassword = bcrypt.hashSync(req.body.password, 8);  
  
  User.create({  
    name : req.body.name,  
    email : req.body.email,  
    password : hashedPassword  
  },  
  function (err, user) {  
    if (err) return res.status(500).send("There was a problem registering the user`.");  
  
    // if user is registered without errors  
    // create a token  
    var token = jwt.sign({ id: user._id }, config.secret, {  
      expiresIn: 86400 // expires in 24 hours  
    });  
  
    res.status(200).send({ auth: true, token: token });  
  });  
}  
  
authController.profile = function(req, res, next) {  
  
  User.findById(req.userId, function (err, user) {  
    if (err) return res.status(500).send("There was a problem finding the user.");  
    if (!user) return res.status(404).send("No user found.");  
    res.status(200).send(user);  
  });  
}
```

NodeJS e ExpressJS - Autenticação & Autorização

Controlador de
usado para gerir a
autenticação e
autorização

(continuação..)

```
//middleware
authController.verifyToken = function(req, res, next) {

  // check header or url parameters or post parameters for token
  var token = req.headers['x-access-token'];
  if (!token)
    return res.status(403).send({ auth: false, message: 'No token provided.' });

  // verifies secret and checks exp
  jwt.verify(token, config.secret, function(err, decoded) {
    if (err)
      return res.status(500).send({ auth: false, message: 'Failed to authenticate token.' });

    // if everything is good, save to request for use in other routes
    req.userId = decoded.id;
    next();
  });
}

authController.verifyRoleAdmin = function(req, res, next) {

  User.findById(req.userId, function (err, user) {
    if (err) return res.status(500).send("There was a problem finding the user.");
    if (!user) return res.status(404).send("No user found.");
    if (user.email === 'me@estg.ipp.pt'){
      next();
    } else {
      return res.status(403).send({ auth: false, message: 'Not authorized!' });
    }
  });
}

module.exports = authController;
```

NodeJS e ExpressJS - Autenticação & Autorização

Router

Autenticação

Autorização

```
var express = require('express');
var router = express.Router();
var authController = require('../controllers/authController');
var userController = require('../controllers/userController');

router.post('/login', authController.login);
router.post('/register', authController.register);
router.get('/logout', authController.logout);
router.get('/profile', authController.profile);

router.get('/all-users', authController.verifyToken, authController.verifyRoleAdmin, userController.getUsers);

module.exports = router;
```



Angular - Autenticação & Autorização

Do lado de Angular usamos os serviços de autenticação definidos:

- Verificar autenticação com serviços REST
- Guardar token de autenticação no local storage
- Para rotas protegidas verificar se existe o token do utilizador no local storage
- Usar o token no local storage para pedidos futuros à API REST

Verificar permissões com serviços que implementam a interface CanActivate e routing da aplicação

- Verificar em cada endereço protegido se o utilizador está autenticado

Angular Autenticação

```
import { Injectable } from '@angular/core';
import { Router, CanActivate, ActivatedRouteSnapshot, RouterStateSnapshot } from '@angular/router';

@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {

  constructor(private router: Router) { }

  canActivate(route: ActivatedRouteSnapshot, state: RouterStateSnapshot) {
    if (localStorage.getItem('currentUser')) {
      // logged in so return true
      return true;
    }

    // not logged in so redirect to login page with the return url
    this.router.navigate(['/login'], { queryParams: { returnUrl: state.url } });
    return false;
  }
}
```


Angular Autenticação

Para endereços não autenticados que não passam pela guard devemos reencaminhar para a página de login

Devemos editar o ficheiro de routing

Especificar o parâmetro “canActivate” que é verificado antes de entrar no endereço definido

Angular Autenticação

```
import { Routes, RouterModule } from '@angular/router';

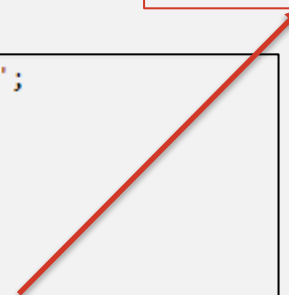
import { HomeComponent } from './home';
import { LoginComponent } from './login';
import { AuthGuard } from './_guards';

const appRoutes: Routes = [
  { path: '', component: HomeComponent, canActivate: [AuthGuard] },
  { path: 'login', component: LoginComponent },

  // otherwise redirect to home
  { path: '**', redirectTo: '' }
];

export const routing = RouterModule.forRoot(appRoutes);
```

Rota protegida



Angular Autenticação

Interceptors

- Podemos usar Interceptors para adicionar o token de autenticação a pedidos http
- Devemos declarar os interceptors no ficheiro “app.module.ts” nas parte providers

Angular Autenticação

```
import { Injectable } from '@angular/core';
import { HttpRequest, HttpHandler, HttpEvent, HttpInterceptor } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable()
export class JwtInterceptor implements HttpInterceptor {
  intercept(request: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    // add authorization header with jwt token if available
    let currentUser = JSON.parse(localStorage.getItem('currentUser'));
    if (currentUser && currentUser.token) {
      request = request.clone({
        setHeaders: {
          Authorization: `Bearer ${currentUser.token}`
        }
      });
    }

    return next.handle(request);
  }
}
```

Angular Autenticação

```
import { AppComponent } from './app.component';
import { routing } from './app.routing';

import { JwtInterceptor, ErrorInterceptor } from './_helpers';
import { HomeComponent } from './home';
import { LoginComponent } from './login';

@NgModule({
  imports: [
    BrowserModule,
    ReactiveFormsModule,
    HttpClientModule,
    routing
  ],
  declarations: [
    AppComponent,
    HomeComponent,
    LoginComponent
  ],
  providers: [
    { provide: HTTP_INTERCEPTORS, useClass: JwtInterceptor, multi: true },
```

Angular Autenticação

Serviço de autenticação

- Podemos encapsular a autenticação em serviços REST a uma API do lado do servidor
- REST API desenvolvida com o módulo JWT para autenticação

Angular Autenticação

```
@Injectable({
  providedIn: 'root'
})
export class AuthenticationService {

  constructor(private http: HttpClient) { }

  login(email: string, password: string) : Observable<any> {
    return this.http.post<any>('http://localhost:3000/api/v1/auth/login', { email, password });
  }

  logout() {
    // remove user from local storage to log user out
    localStorage.removeItem('currentUser');
  }

  register(username: string, password: string) : Observable<any>{
    return this.http.post<any>('http://localhost:3000/api/v1/auth/users/register', { username, password })
  }
}
```

Angular Autenticação

Adicionar verificações às
nossas páginas protegidas

No exemplo:

- Todos podem consultar listagem de produtos
- Só utilizadores autenticados podem aceder a detalhes ou adicionar novos products

```
const routes: Routes = [
  {
    path: 'products',
    component: ProductListComponent
  },
  {
    path: 'product-add',
    component: ProductAddComponent,
    canActivate: [AuthGuardService]
  },
  {
    path: 'product-detail/:id',
    component: ProductDetailComponent,
    canActivate: [AuthGuardService]
  },
  {
    path: 'login',
    component: LoginComponent
  },
  // default redirect to home
  { path: '**', redirectTo: 'products' }
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```


Angular Components

Um dos pontos fortes de Angular está na reutilização de componentes

Podemos inclusive passar dados de uns componentes para os outros utilizando o selector do componente e atributos com a tag `@Input()`

Angular Components

Juntar vários componentes numa única página e reutiliza-los

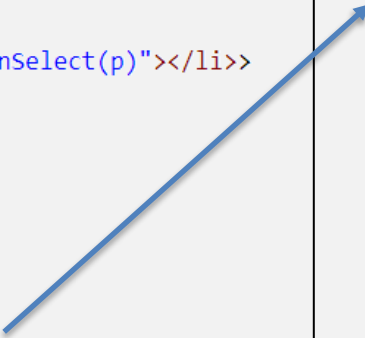
```
<h2>Product List</h2>

<div>
  <button (click)="add()">
    Add
  </button>
</div>

<ul class="products">
  <li *ngFor="let p of products; let i=index;" (click)="onSelect(p)"></li>>
    <a routerLink="/product-details/{{p._id}}">
      <span class="badge">{{i+1}}</span> {{p.name}}
    </a>
    <button class="delete" title="delete product"
      (click)="delete(p._id)">x</button>
    </li>
</ul>

<app-product-detail [product]="selectedProduct"></app-product-detail>
```

Componente de
detalhes



Angular Components

Juntar vários componentes numa única página e reutiliza-los

```
@Component({  
  selector: 'app-product-list',  
  templateUrl: './product-list.component.html',  
  styleUrls: ['./product-list.component.css']  
})  
export class ProductListComponent implements OnInit {  
  
  products:any = [];  
  rest:RestService;  
  selectedProduct:Product;  
  
  constructor(rest:RestService, private route: ActivatedRoute, private router: Router) {  
    this.rest = rest;  
  }  
  
  ngOnInit() {  
    this.getProducts();  
  }  
}
```

Produto para o
componente de
detalhes

Angular Components

Juntar vários componentes numa única página e reutiliza-los

```
@Component({  
  selector: 'app-product-detail',  
  templateUrl: './product-detail.component.html',  
  styleUrls: ['./product-detail.component.css']  
})  
export class ProductDetailComponent implements OnInit {  
  
  @Input() product : Product;  
  
  constructor(public rest:RestService, private route: Acti  
  }  
}
```

Produto para o
componente de
detalhes

```
<app-product-detail [product]="selectedProduct"></app-product-detail>
```

Angular Components

Podemos utilizar serviços para guardar informação que queremos passar de um componente para o outro

O estado dos dados é guardado no serviço

Angular Components

Demonstração

Angular Material

É uma biblioteca de componentes para Angular

Pode ser instalada via comando ng:

- `ng add @angular/material`

Necessário importar os módulos em `app.module.ts`

Angular Material

Necessário importar os módulos em app.module.ts

Exemplo:

```
import {MatCardModule} from '@angular/material/card';
import { MatInputModule } from '@angular/material/input';
import { MatPaginatorModule } from '@angular/material/paginator';
import { MatProgressSpinnerModule } from '@angular/material/progress-spinner';
import { MatSortModule } from '@angular/material/sort';
import { MatTableModule } from '@angular/material/table';
import { MatIconModule } from '@angular/material/icon';
import { MatButtonModule } from '@angular/material/button';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatSliderModule } from '@angular/material/slider';
import { MatSlideToggleModule } from '@angular/material/slide-toggle';
import { MatButtonToggleModule } from '@angular/material/button-toggle';
import { MatSelectModule } from '@angular/material/select';
```


Angular Material

Necessário importar os módulos em app.module.ts

Exemplo:

```
/// ...  
imports: [  
  BrowserModule,  
  AppRoutingModule,  
  HttpClientModule,  
  FormsModule,  
  BrowserAnimationsModule,  
  MatInputModule,  
  MatPaginatorModule,  
  MatProgressSpinnerModule,  
  MatSortModule,  
  MatTableModule,  
  MatIconModule,  
  MatButtonModule,  
  MatCardModule,  
  MatFormFieldModule,  
  MatSliderModule,  
  MatSlideToggleModule,  
  MatButtonModuleToggleModule,  
  MatSelectModule  
],
```

Angular Material

Usar diretamente nos componentes de angular

```
@Component({
  selector: 'app-login',
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.css']
})
export class LoginComponent implements OnInit {

  username: string;
  password: string;

  constructor(private router: Router, private authService: AuthenticationService) { }

  ngOnInit(): void {
  }

  login() : void {
    this.authService.login(this.username, this.password).subscribe((user : any)=>{
      if (user && user.token) {
        // store user details and jwt token in local storage to keep user logged in between page refreshes
        localStorage.setItem('currentUser', JSON.stringify(user));
        this.router.navigate(['/productlist']);
      }
    })
  }
}
```

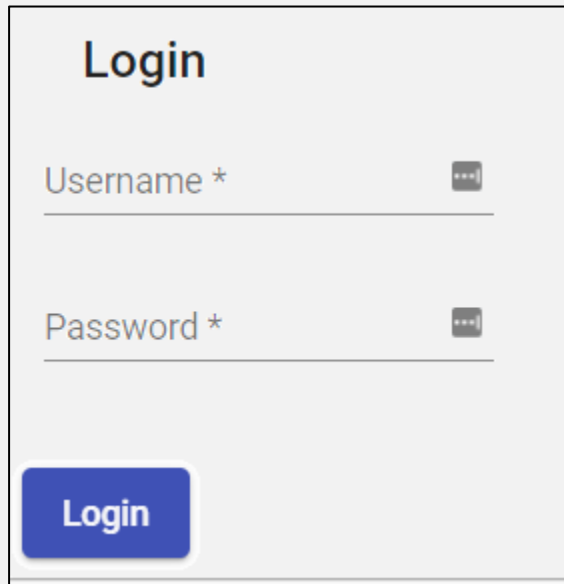
Angular Material

Usar diretamente nos componentes de angular

```
<mat-card class="example-card">
  <mat-card-header>
    <mat-card-title>Login</mat-card-title>
  </mat-card-header>
  <mat-card-content>
    <form class="example-form">
      <table class="example-full-width" cellspacing="0">
        <tr>
          <td>
            <mat-form-field class="example-full-width">
              <input matInput placeholder="Username" [(ngModel)]="username" name="username" required>
            </mat-form-field>
          </td>
        </tr>
        <tr>
          <td>
            <mat-form-field class="example-full-width">
              <input matInput placeholder="Password" [(ngModel)]="password" type="password"
                name="password" required>
            </mat-form-field>
          </td>
        </tr>
      </table>
    </form>
  </mat-card-content>
  <mat-card-actions>
    <button mat-raised-button (click)="login()" color="primary">Login</button>
  </mat-card-actions>
</mat-card>
```

Angular Material

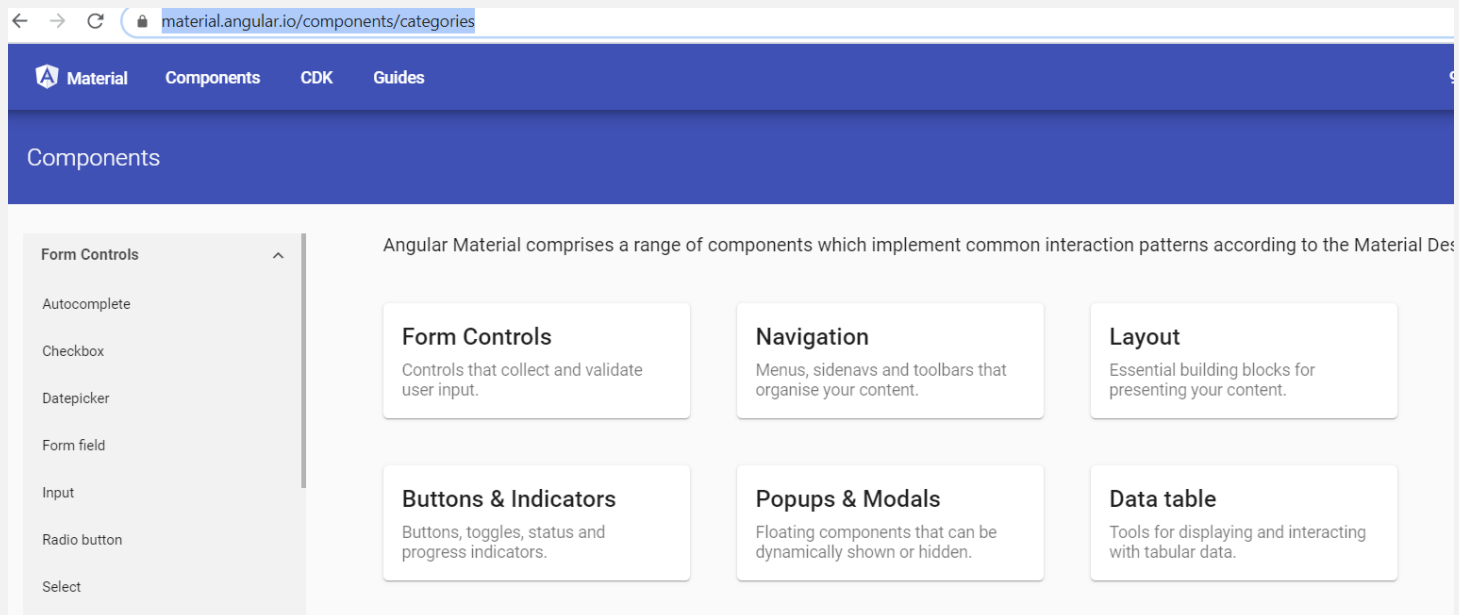
Usar diretamente nos componentes de angular



A screenshot of an Angular Material login form. The form is titled "Login" in a bold, dark font. It contains two input fields: "Username *" and "Password *". Each input field has a small, dark, rounded rectangle icon to its right, which typically represents a password toggle or a clear button. Below the input fields is a blue button with the text "Login" in white. The entire form is enclosed in a thin black border.

Angular Material

Mais componentes e exemplos em
<https://material.angular.io/components/categories>



Referências

JSONWebToken:

- <https://www.npmjs.com/package/jsonwebtoken>

Angular:

- <https://angular.io/api/common/http/HttpInterceptor>
- <https://angular.io/api/router/CanActivate>
- <https://angular.io/start/start-data>

Angular Material:

- <https://material.angular.io/>

P.PORTO

Angular

Programação em Ambiente Web